

Pergunta 1

0 / 0,3 pontos

Um sistema de *software* envolve projetos com diferentes escopos, permitindo que as tecnologias e soluções necessárias sejam interligadas formando um grande conjunto de dados, a serem posteriormente vinculados à interface do usuário e à arquitetura.

No que se refere aos tipos de projeto, avalie as asserções a seguir e a relação proposta entre elas.

I. Um projeto de componentes e módulos é essencial para entregar a modularidade de um sistema. Os componentes e módulos são segmentados para definir a base estrutural do *software*

PORQUE

II. É através dos componentes e módulos que os algoritmos são desenvolvidos com o propósito de executar funções ou tarefas específicas.

A respeito dessas asserções, assinale a opção correta:

- ☒ ☐ As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- ☐ ☒ As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- ☐ A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- ☐ A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- ☐ As asserções I e II são proposições falsas.

Pergunta 2

0,3 / 0,3 pontos

Em um sistema devem ser definidos os requisitos funcionais e não funcionais de um sistema, que garantem que o projeto atenda às necessidades planejadas e ofereça os recursos necessários para que os usuários possam executar suas tarefas.

No que se refere aos requisitos funcionais e não funcionais, analise as afirmações a seguir.

I. Os requisitos não funcionais prezam pela qualidade, desempenho e restrições de um sistema.

II. Os requisitos funcionais são essenciais para que o sistema funcione corretamente e definem o seu comportamento.

III. As respostas do sistema são resultantes dos requisitos não funcionais e por isso afetam diretamente as ações do usuário.

IV. Os requisitos funcionais interpretam todas as entradas recebidas pelo sistema e definem como as tarefas e processos ocorrem.

É correto o que se afirmar em:

- ☐ I e II, apenas.
- ☒ I, II e IV, apenas.
- ☐ I, II e III, apenas.
- ☐ II e IV, apenas.
- ☐ III, apenas.

Pergunta 3

0,3 / 0,3 pontos

Os diagramas são ferramentas diretamente relacionadas com a arquitetura e os padrões de *softwares*, utilizados para modelar e desenvolver aplicações, facilitar a visualização dos elementos e simplificar o entendimento dos projetos. Existem diferentes tipos de diagramas e o modelo geralmente utilizado para facilitar a visualização lógica de um projeto é o

- ✓ ☒ diagrama de sequência.
- ☐ diagrama de codificação.
- ☐ diagrama de componentes.
- ☐ diagrama de implementação.
- ☐ diagrama de pacotes.

Pergunta 4

0 / 0,3 pontos

U012025.2FA-01F

Os padrões de projeto devem ser utilizados sempre com foco no problema que precisa ser resolvido, e não apenas aplicados de forma mecânica ou decorativa. Eles funcionam como soluções testadas e validadas para desafios recorrentes no desenvolvimento de software, oferecendo estruturas que tornam o código mais organizado, reutilizável e fácil de manter.

É importante compreender que cada padrão foi criado para lidar com uma situação específica: por exemplo, padrões de criação (como Singleton ou Factory Method) auxiliam no gerenciamento de instâncias de objetos, enquanto os padrões estruturais (como Adapter ou Composite) organizam melhor a composição de classes e objetos. Já os padrões comportamentais (como Observer ou Strategy) lidam com a comunicação e o comportamento entre diferentes partes do sistema.

Assinale a opção correta:

- ☐ Sistema embarcados nunca devem padrões de projeto.
- ✗ ☒ a escolha de qual padrão adotar deve levar em conta o contexto do problema, os requisitos do sistema e a complexidade da solução.
- ☐ Sistemas embarcados utilizam apenas códigos simples e não precisam de padrões de projeto.
- ⇒ ☐ Os padrões de projeto devem sempre ser aplicada em projetos de alta complexidade
- ☐ O uso adequado de padrões de projeto pode até prejudicar o projeto, tornando-o mais difícil de entender ou manter. Um padrão de projeto deve ser adequado ao problema a ser resolvido.

▼ [Ocultar feedback da pergunta 4](#)

Feedback

Os padrões de projeto podem ser utilizados, sempre adequados ao problema a ser resolvido.

Pergunta 5

0,3 / 0,3 pontos

A criação da documentação é uma fase importante para um projeto de *software*. A documentação precisa estar devidamente atualizada e pode ser construída através de metodologias que permitam implementar normas e padrões adequados. Da mesma forma, o *design* de *software* é um processo iterativo que geralmente envolve o refinamento dos requisitos e é uma fase tão importante quando a fase de registro.

No que diz respeito às fases de documentação e design, avalie as asserções a seguir e a relação proposta entre elas.

I.

Ao documentar as decisões a serem tomadas no processo de construção de um *software*, informações relevantes devem ser registradas e sempre analisadas antes de implementar qualquer funcionalidade.

PORQUE

II. Ao manter o registro do que deve ser executado em um projeto, o *design* também pode ser beneficiado com melhorias sempre que novas necessidades forem identificadas e possam ser adaptadas à estrutura existente.

A respeito dessas asserções, assinale a opção correta:

- ☐ As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- ☐ As asserções I e II são proposições falsas.
- ☒ As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- ☐ A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- ☐ A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.

Pergunta 1

0,3 / 0,3 pontos

Um Padrão (*Pattern*) é uma solução recorrente para resolver determinado problema e sua estrutura é formada por quatro elementos essenciais, cada um deles capaz de cumprir determinada função. Sobre os elementos essenciais de um Padrão de *software*, avalie as afirmativas a seguir como verdadeiras (V) ou falsas (F).

- () Uma das principais metas do *DevOps* é promover a colaboração estreita entre as equipes de desenvolvimento e operações, resultando em um ciclo de desenvolvimento mais rápido e na entrega contínua de *software* de alta qualidade.
- () Os princípios-chave do *DevOps* incluem uma cultura de automação, a aplicação de princípios *Lean IT*, a medição constante do desempenho e a promoção do compartilhamento de conhecimento entre as equipes.
- () O compartilhamento dos dados em um projeto é um princípio básico no entanto deve ser praticado somente por membros de uma mesma equipe, para facilitar as tarefas a serem executadas.
- () Os processos automatizados são executados com auxílio de computadores e ferramentas tecnológicas programadas para executar principalmente tarefas repetitivas, minimizando erros de operação.
- () A integração da equipe permite que um único teste seja realizado ao final do projeto, atendendo aos requisitos de qualidade mapeados e agilizando as correções que precisam ser aplicadas.

A sequência correta, de cima para baixo, é:

- ☐ V, F, F, V, F.
- ☒ V, V, F, V, F.
- ☐ V, F, F, V, V.
- ☐ F, V, V, V, F.
- ☐ V, V, F, V, V.

Pergunta 2

0 / 0,3 pontos

A relação entre modelos e padrões de software é fundamental para o desenvolvimento eficiente e sustentável de sistemas de software. Ambos desempenham papéis cruciais em diferentes etapas do ciclo de vida de desenvolvimento de software e estão interligados de várias maneiras, por isso é importante conhecer as características de cada um dos modelos, facilitando a forma de integração com padrões de *softwares* adequados.

Considerando as informações apresentadas, avalie as afirmativas a seguir como verdadeiras (V) ou falsas (F).

- () O modelo espiral é aberto à mudanças constantes e foi criado exatamente para entregar maior eficiência e qualidade em comparação a seus antecessores. Versões completas de *software* podem ser desenvolvidas com agilidade, mas também é compatível com padrões de *software*.
- () O modelo cascata foi o primeiro a surgir e apesar de ter sido um grande aliado na condução de projetos de *software*, atualmente não é tão utilizado pela sua rigidez e dificuldade de moldagem em relação aos padrões reutilizáveis.
- () Os padrões de *softwares reutilizáveis* surgiram para evitar que as equipes precisam criar um *software* do zero e substituíram totalmente todos os modelos de *software* que eram implementados anteriormente.
- () Os padrões de *software* são considerados como uma caixa de ferramentas valiosa para equipes que adotam um modelo iterativo de desenvolvimento, por permitir a solução de problemas recorrentes e permitir mudanças contínuas durante o processo iterativo.
- () Modelos engessados que não permitem alterar fases anteriores do projeto são mais rígidos e por isso são os mais adotados, por evitar que o projeto seja estendido por mais tempo, minimizando problemas operacionais.

A sequência correta, de cima para baixo, é:

- ☐ F, V, F, F, V.
- ☐ V, V, V, V, V.
- ☒ F, V, F, V, F.
- ☒ V, V, F, V, F.
- ☐ V, V, F, V, V.

Pergunta 3

0,3 / 0,3 pontos

Apesar de os *softwares* serem definidos de forma genérica como programas de computador executados com base em um conjunto de instruções ou dados que permitem que um computador execute tarefas específicas, podem ser categorizados em diferentes tipos, que define sua capacidade de controlar e coordenar as atividades com base em seus componentes e módulos.

A respeito dos tipos de *software*, analise as afirmações a seguir.

- I. Os *softwares* de aplicativos são construídos para serem utilizados por desenvolvedores e usuários com conhecimento em programação, por serem de alta complexidade e executados com base em instruções inseridas no código-fonte.
- II. Os *softwares* de programação são construídos com base em ambientes de desenvolvimento integrados (IDEs), compiladores, editores de código, depuradores e ferramentas avançadas que auxiliam os desenvolvedores.
- III. Os *softwares* de sistema são considerados obsoletos por serem compatíveis apenas com plataformas de sistema operacional limitadas e não oferecerem o suporte necessário ao usuário.
- IV. Os *softwares* embarcados são aqueles que são comercializados em com dispositivos eletrônicos específicos e permitem sua funcionalidade, apesar da limitação de operar em determinadas plataformas.

É correto o que se afirmar em:

- ☐ I apenas.
- ☐ II, III e IV apenas.
- ☒ II e IV apenas.
- ☐ III e IV, apenas.
- ☐ I, II e IV, apenas.

Pergunta 4

0,3 / 0,3 pontos

A entrega contínua em projetos de software envolve diferentes sistemas, metodologias e recursos que se encaixam no ciclo de desenvolvimento, facilitando que a equipe possa pensar de maneira integrada e ao mesmo tempo executar tarefas independentes. Com relação à cultura DevOps e as ferramentas de apoio no desenvolvimento de softwares, avalie as afirmativas a seguir como verdadeiras (V) ou falsas (F).

() O Git é um sistema de controle de versão distribuído que permite o acompanhamento das mudanças feitas em arquivos ao longo do tempo, sendo amplamente utilizado para o desenvolvimento de software.

() Por ser um servidor central, o Git contém todos os arquivos de um projeto que são diretamente acessados e compartilhados na mesma estrutura, por isso o Git também cumpre a função de repositório.

() Os colaboradores podem selecionar em que versão do repositório irão atuar, através de uma operação denominada checkout, neste caso todos os arquivos são atualizados quando o recurso é utilizado.

() Apesar de o Git e o Github serem tecnologias que auxiliam na colaboração entre equipes e serem utilizadas em conjunto com DevOps, a cultura define a colaboração entre equipes para entregar software de maneira mais rápida e confiável.

() O GitHub é uma plataforma de hospedagem de código fonte que utiliza o sistema de controle de versão Git, permitindo que equipes colaborem no desenvolvimento de software, revisem e discutam código, e acompanhem as alterações ao longo do tempo.

A sequência correta, de cima para baixo, é:

- ☐ V, F, V, V, F.
- ☐ F, V, V, F, F.
- ☐ V, V, F, V, V.
- ☐ F, V, F, V, F.
- ☒ V, F, V, V, V.

Pergunta 5

0,3 / 0,3 pontos

U022024.1JP-01M

Durante o desenvolvimento de software, é essencial que todas as etapas do ciclo de vida sejam rigorosamente cumpridas para garantir a qualidade e a funcionalidade final do sistema. Cada uma dessas etapas tem um papel específico e contribui para a produção de um software eficiente e bem estruturado. Com base nos processos do ciclo de vida de desenvolvimento de software, qual das etapas abaixo deve ser revisada cuidadosamente para garantir que o software atenda plenamente às necessidades dos usuários e evite problemas nas etapas subsequentes?

- ☐ Suporte e Manutenção
- ☐ Teste.
- ☐ Implementação.
- ✓ ☒ Levantamento de requisitos.
- ☐ Arquitetura de software.

▼ [Ocultar feedback da pergunta 5](#)

Feedback

O levantamento de requisitos é a etapa inicial em que se identificam e documentam as necessidades e expectativas do usuário, servindo como base para todo o desenvolvimento. Um levantamento inadequado pode comprometer todo o projeto.

Pergunta 1

0,3 / 0,3 pontos

As ferramentas de análise de requisitos têm o propósito de ajudar na captura, análise, documentação e gerenciamento dos requisitos de um projeto ou de um sistema de *software*. Estas ferramentas auxiliam na compreensão, rastreamento e implementação dos requisitos de forma adequada. A *Jama Connect* e a *IBM Engineering Management DOORS* são exemplos de ferramentas utilizadas nesse processo.

Sobre as ferramentas *Jama Connect* e *IBM Engineering Management DOORS*, assinale a alternativa correta.

- ✓ ☒ A ferramenta *Jama Connect* é usada no rastreamento de requisitos de *software* e fornece uma interface intuitiva.
- ☐ A *IBM Engineering Requirements Management DOORS* é usada no gerenciamento de requisitos simples.
- ☐ A *IBM Engineering Requirements Management DOORS* tem como ponto negativo não otimizar a comunicação.
- ☐ A *Jama Connect* tem como uma de suas desvantagens a falta de transparência multifuncional.
- ☐ capacidade de revisão e aprovação de requisitos da *Jama Connect* é considerada bastante complexa.

Pergunta 2

0,3 / 0,3 pontos

A qualidade do *software* poderá ser comprometida pela desorganização de *software*, sendo esta causada por diversas razões. Os testes de qualidade têm grande importância para que sejam detectados ou evitados defeitos que danifiquem o *software*. Dentre os fatores que interferem na qualidade de *software* temos a usabilidade aprimorada, economia de recursos e a segurança e proteção de dados. Sobre estes fatores, avalie as afirmativas a seguir como verdadeiras (V) ou falsas (F).

- () A identificação antecipada de defeitos através dos testes de qualidade pode gerar importante economia de recursos.
- () Os testes de qualidade poderão detectar vulnerabilidades que colocariam em risco os dados sensíveis.
- () Os testes de usabilidade são responsáveis pela criação de subconjuntos de *softwares* inacessíveis ao usuário.
- () A interface do usuário pode ser aperfeiçoada e incrementada por meio da aplicação dos testes de usabilidade.
- () Os projetos de uma escala maior não sofrem alteração em seus recursos com o uso de testes de qualidade.

A sequência correta, de cima para baixo, é:

- ☐ F, V, F, F, V.
- ☐ V, F, V, F, F.
- ☐ F, F, V, V, V.
- ☐ V, F, F, V, F.
- ☒ V, V, F, V, F.

Pergunta 3

0,3 / 0,3 pontos

As ferramentas de testes de automação de desempenho possibilitam às equipes de teste o reconhecimento de gargalos e problemas de conectividade, para que assim possam otimizar o desempenho. Os problemas relacionados com o desempenho poderão interferir de forma negativa no ambiente de produção. Dentre as ferramentas disponíveis para os testes de automação de desempenho, temos o *Artillery*.

Com relação às características da ferramenta *Artillery*, assinale a alternativa correta.

- ☐ A ferramenta *Artillery* foi criada pela *Apache*, que a transformou em código fechado.
- ☐ O *Artillery* é uma ferramenta de código fechado utilizada para testes de carga.
- ☒ O *Artillery* é planejado para os cenários de testes de desempenho que se baseiam em YAML.
- ☐ A ferramenta *Artillery* foi criada para suportar somente testes de estresse.
- ☐ O *Artillery* possibilita somente os testes de desempenho e confiabilidade.

U042024.1JP-01 M

Em projetos de desenvolvimento de software, a qualidade é um aspecto crucial que permeia todas as fases do ciclo de vida do produto. Desde a concepção até a entrega final, garantir a qualidade é essencial para atender às expectativas dos usuários e manter a competitividade no mercado. Testes de qualidade desempenham um papel fundamental nesse processo, permitindo identificar e corrigir defeitos antes que cheguem aos clientes, garantindo assim a eficácia e confiabilidade do software desenvolvido. Nesse contexto, avalie as asserções a seguir e a relação proposta entre elas.

I. A qualidade em projetos de software é crucial, pois influencia diretamente na satisfação dos usuários e na competitividade no mercado.

PORQUE

II. Os testes de qualidade são indispensáveis para identificar e corrigir defeitos antes da entrega do software aos clientes, garantindo sua eficácia e confiabilidade.

A respeito dessas asserções, assinale a opção correta:

- ☐ A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- ☐ As asserções I e II são proposições falsas.
- ☒ As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- ☐ As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- ☐ A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.

U042024.2JP-01M

No desenvolvimento de software, adotar técnicas de teste é essencial para garantir a qualidade e o sucesso do projeto. Essas técnicas permitem identificar falhas nas etapas iniciais do desenvolvimento, evitando problemas futuros e frustrações por parte dos clientes. Organizações como a IEC/ISO têm estabelecido normas que auxiliam na padronização dos testes e na integração de sistemas, assegurando a segurança e eficiência dos produtos. Além disso, o uso de técnicas consolidadas contribui para a otimização dos recursos e a redução de manutenções, melhorando a entrega final do produto ao mercado (Aniche, 2015). Com base no texto, assinale a alternativa que corretamente identifica uma vantagem de utilizar técnicas de teste de software consolidadas.

- ☒  Reduzem a necessidade de manutenções em etapas iniciais do ciclo de desenvolvimento, melhorando o planejamento dos recursos.
- ☐ Aumentam a complexidade do processo de desenvolvimento, pois as técnicas consolidadas requerem mais tempo de execução.
- ☐ São aplicáveis apenas para grandes projetos, pois os pequenos projetos não se beneficiam das técnicas de teste de software.
- ☐  Permitem que todas as falhas sejam detectadas apenas após a fase de implementação do software.
- ☐ Garantem que o software será livre de falhas após sua implementação.

▼ [Ocultar feedback da pergunta 5](#)

Feedback

As técnicas de teste bem aplicadas ajudam a identificar falhas nas etapas iniciais, o que reduz manutenções e otimiza recursos ao longo do ciclo de desenvolvimento.